



Carátula para entrega de prácticas

Facultad de Ingeniería

Laboratorios de docencia

Laboratorio de Computación Salas A y B

Profesor(a): Rene Adrián Davila Pérez

Asignatura: Programación Orientada a Objetos

Grupo: 7

No de Práctica(s): 4

Integrante(s): 323001315, 426057857, 323008754, 426058665, 323067267

*No. de lista o
brigada:*

5

Semestre: 2027-1

Fecha de entrega: 25 de Septiembre

Observaciones:

CALIFICACIÓN: _____

Índice

1. Introducción	2
2. Marco Teórico	2
2.1. Recolector de basura (Garbage Collector)	2
2.2. Interfaz AutoCloseable y try-with-resources	3
2.3. HashMap	3
2.4. TreeSet	3
2.5. Manejo de cadenas y expresiones regulares	3
3. Desarrollo	3
3.1. Ejercicio 1. destructores en Java	3
3.2. Ejercicio 2. Palabras duplicadas	4
3.3. Pruebas de Ejecución	5
4. Resultados	6
4.1. Resultado ejercicio 1: Investigación LLM	6
4.2. Resultado ejercicio 2: Analizador Palabras	7
5. Conclusiones	8

1. Introducción

- **Planteamiento del problema.** En esta práctica se abordan dos problemáticas relacionadas con el manejo de recursos y datos en el lenguaje Java. Por un lado, existe la necesidad de comprender cómo Java gestiona la liberación de memoria y recursos de un objeto una vez que este deja de ser utilizado, ya que, a diferencia de otros lenguajes como C++, Java no cuenta con un mecanismo explícito de destructores, lo cual genera dudas frecuentes sobre cómo se controla el ciclo de vida de los objetos.

Por otro lado, se plantea un problema de procesamiento de texto: dada una oración, se requiere determinar cuántas palabras se repiten dentro de ella, considerando que las mayúsculas y minúsculas deben tratarse como equivalentes y que los signos de puntuación no deben afectar el conteo.

- **Motivación.** Es necesario dar solución a estos problemas porque ambos representan situaciones que un programador enfrenta con frecuencia en el desarrollo de software. Comprender el manejo de memoria en Java (a través del Garbage Collector) en lugar de destructores explícitos es fundamental para escribir código eficiente y evitar fugas de recursos, especialmente al trabajar con archivos, conexiones de red o bases de datos. Asimismo, el conteo de palabras duplicadas es una habilidad práctica aplicable en áreas como el procesamiento de lenguaje natural, la validación de textos y el análisis de datos, por lo que dominar la manipulación de cadenas de texto (normalización, limpieza de puntuación y comparación) resulta indispensable para cualquier desarrollador.
- **Objetivos.**
 - Comprender el concepto de destructores y contrastarlo con el modelo de gestión automática de memoria que utiliza Java mediante el Garbage Collector.
 - Identificar los mecanismos que ofrece Java para el manejo explícito de recursos externos, como try-with-resources y la interfaz AutoCloseable.
 - Diseñar e implementar una aplicación en Java capaz de procesar una oración, normalizar su texto (ignorando mayúsculas/minúsculas y signos de puntuación) y determinar el número de palabras duplicadas.
 - Validar el correcto funcionamiento del programa mediante pruebas con distintas entradas de texto.

2. Marco Teórico

2.1. Recolector de basura (Garbage Collector)

A diferencia de lenguajes como C++, Java no cuenta con destructores en el sentido tradicional. La liberación de memoria se delega a un proceso automático llamado

Recolector de Basura (Garbage Collector o GC), que se ejecuta en segundo plano y elimina los objetos que ya no tienen ninguna referencia apuntándolos [1].

2.2. Interfaz AutoCloseable y try-with-resources

Para liberar recursos externos (archivos, conexiones a red, bases de datos), Java utiliza la interfaz AutoCloseable junto con la estructura try-with-resources, la cual garantiza que el método close() se invoque automáticamente al salir del bloque, incluso si ocurre una excepción [1].

2.3. HashMap

Es una estructura de datos que almacena información en pares clave-valor. Utiliza una función hash para ubicar los elementos, lo que permite accesos casi instantáneos ($O(1)$), aunque no mantiene ningún orden entre ellos [2].

2.4. TreeSet

Es una estructura que almacena elementos únicos dentro de un árbol, ordenándolos automáticamente conforme se insertan (alfabética o numéricamente), con un tiempo de acceso de $O(\log n)$ [1].

2.5. Manejo de cadenas y expresiones regulares

La clase String permite transformar texto mediante métodos como toLowerCase() y trim(). Además, el método replaceAll() utiliza expresiones regulares para eliminar patrones de caracteres, lo cual es útil para quitar signos de puntuación antes de procesar una oración [1].

3. Desarrollo

3.1. Ejercicio 1. Destructores en Java

1. **Inicio de la investigación en equipo.** Cada integrante consultó de forma independiente a su propio LLM sobre el concepto y funcionamiento de los destructores en Java.
2. **Consulta del Integrante 1.** Se preguntó por qué Java no implementa destructores como C++ y cómo gestiona la memoria en su lugar.
3. **Consulta del Integrante 2.** Se investigó el funcionamiento del Recolector de Basura (*Garbage Collector*) y el criterio que utiliza para determinar cuándo un objeto es elegible para ser eliminado.
4. **Consulta del Integrante 3.** Se indagó sobre el método finalize(), su propósito original y las razones por las que fue marcado como obsoleto.

5. **Consulta del Integrante 4.** Se preguntó cómo se liberan recursos externos (archivos, conexiones) si el GC solo gestiona memoria RAM.
6. **Consulta del Integrante 5.** Se investigó la interfaz `AutoCloseable` y la estructura `try-with-resources` como mecanismo moderno de liberación de recursos.
7. **Comparación de resultados.** Se contrastaron las cinco investigaciones para identificar coincidencias sobre el manejo automático de memoria en Java.
8. **Selección.** Se integró una única respuesta consolidada, complementada con los aportes de cada integrante, para incluirla en el Marco Teórico y las Conclusiones.

3.2. Ejercicio 2. Palabras duplicadas

1. **Inicio del programa.** Se crea un objeto de la clase `AnalizadorPalabras`, el cual recibe como parámetro la oración a analizar y crea internamente un `HashMap` vacío donde se almacenará la frecuencia de cada palabra.
2. **Normalización del texto.** Se convierte toda la oración a minúsculas y, mediante una expresión regular, se eliminan los signos de puntuación y caracteres no alfabéticos, dejando únicamente letras y espacios.
3. **Separación en palabras.** La cadena limpia se divide en un arreglo de palabras utilizando los espacios en blanco como delimitador.
4. **Conteo de frecuencias.** Se recorre el arreglo de palabras y, para cada una, se verifica si ya existe como clave en el `HashMap`: si existe, se incrementa su contador; si no existe, se agrega con valor inicial de uno.
5. **Número de palabras distintas.** Se obtiene consultando el tamaño del `HashMap`, ya que cada clave representa una palabra única.
6. **Número de palabras duplicadas.** Se recorren los valores (frecuencias) del mapa y se cuentan aquellas cuyo valor es mayor a uno.
7. **Despliegue sin ordenar.** Se recorren las claves del `HashMap` en el orden en que fueron almacenadas internamente, mostrando solo aquellas cuya frecuencia es mayor a uno.
8. **Despliegue ordenado.** Las claves del `HashMap` se insertan en un `TreeSet`, el cual las ordena alfabéticamente de forma automática; posteriormente se recorre este conjunto mostrando únicamente las palabras duplicadas.

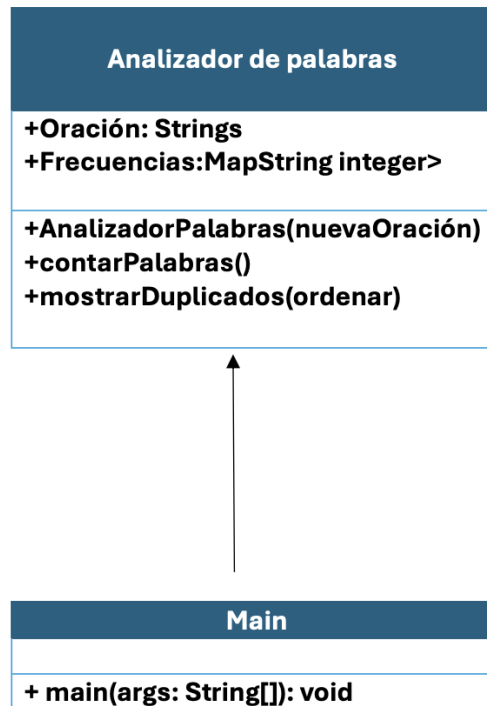


Figura 1: Diagrama de clases ejercicio 2

3.3. Pruebas de Ejecución

Para verificar el correcto funcionamiento de la aplicación del Ejercicio 2, se realizaron pruebas comprobando que se ignoren las mayúsculas y los signos de puntuación:

- a) **Prueba 1: Sensibilidad a mayúsculas y signos básicos.** Se ingresó la oración “Hola mundo. Hola a todos en este mundo!”. El programa reportó un total de 8 palabras, de las cuales 6 son distintas y 2 están duplicadas (*hola*: 2, *mundo*: 2).
- b) **Prueba 2: Múltiples signos de puntuación juntos.** Se ingresó la oración “El perro, el gato y el ratón; corrían rápido. ¿Rápido el perro?”. El programa reportó un total de 12 palabras, de las cuales 7 son distintas y 3 están duplicadas (*el*: 4, *perro*: 2, *rápido*: 2).
- c) **Prueba 3: Ausencia de duplicados.** Se ingresó la oración “Programar en Java es muy interesante.”. El programa reportó un total de 6 palabras, todas distintas, sin ningún duplicado, mostrando el mensaje “No hay palabras duplicadas.”.

4. Resultados

4.1. Resultado ejercicio 1: Investigación LLM

1. En cuanto a la investigación con el LLM sobre los destructores en java, cada integrante preguntó a su LLM, se usó Gemini IA, Claude, Chat GPT, etc.

Al consolidar en una sola idea lo que nos dio a los 5:

En Java no existen los destructores. La limpieza de un programa se maneja en dos frentes:

- **Memoria (Automática):** El Garbage Collector elimina de forma automática los objetos que ya no se usan. El programador no interviene (el antiguo método `finalize()` está obsoleto y causa problemas).
- **Recursos externos (Manual):** Archivos, sockets y bases de datos no se cierran solos. El estándar actual es implementar la interfaz `AutoCloseable` y usar bloques `try-with-resources` para garantizar que el recurso se libere inmediatamente de forma segura.

En Java no existen los destructores tradicionales. La limpieza de una aplicación se divide en dos responsabilidades separadas: la memoria (gestionada automáticamente) y los recursos externos (gestionados por el programador).

Aquí tienes la consolidación de los puntos clave de la investigación:

- **Gestión de Memoria Automática:** El programador no libera memoria manualmente ni decide cuándo se destruye un objeto. El **Garbage Collector (Recolector de Basura)** se ejecuta en segundo plano y elimina de forma impredecible los objetos que ya no tienen referencias apuntando a ellos.
- **El método `finalize()` está obsoleto:** Históricamente se intentó usar este método como un destructor, pero generaba problemas de rendimiento, fugas de memoria y ejecuciones impredecibles. Fue declarado obsoleto a partir de Java 9 y eliminado en versiones recientes.
- **Liberación de Recursos (La alternativa moderna):** Aunque la RAM se limpia sola, los recursos externos (archivos, conexiones a bases de datos, redes) deben cerrarse explícitamente de forma inmediata.
- **Mejores prácticas (`try-with-resources`):** El estándar actual para "destruir" el acceso a un recurso es implementar la interfaz `AutoCloseable` y usar bloques `try-with-resources`. Esto garantiza que el método `close()` se ejecute de forma predecible y automática en cuanto el programa sale del bloque de código, incluso si ocurre un error.
- **API `Cleaner`:** Para casos muy avanzados donde se necesita una red de seguridad por si el desarrollador olvida cerrar un recurso, Java introdujo la clase `Cleaner` (sustituto seguro de `finalize()`).

Figura 2: Resumen destructor LLM.

4.2. Resultado ejercicio 2: Analizador Palabras

2. El programa es una aplicación en Java que solicita al usuario ingresar una oración por consola para analizar su contenido léxico. Su función principal es procesar el texto para extraer estadísticas precisas: cuenta el total de palabras escritas, identifica cuántas son únicas y determina cuántas están duplicadas. Finalmente, el algoritmo calcula la cantidad de veces que se repite cada término y muestra un desglose de las palabras duplicadas, presentándolas primero en una lista sin ordenar y después organizadas de forma alfabética con sus respectivas frecuencias.

```
PS C:\Users\acer\Desktop\POO> javac Main.java AnalizadorPalabras.java
PS C:\Users\acer\Desktop\POO> java Main
Ingresa una oración para analizar: cada paso que doy en este largo camino es un paso nuevo y cada paso nuevo hace que el camino sea menos largo
y paso a paso el camino se acorta hasta que al final del camino un nuevo paso empieza

Total de palabras: 41
Palabras distintas: 24
Palabras diferentes duplicadas: 9

Frecuencia de todas las palabras:
que: 3
hasta: 1
a: 1
camino: 4
el: 2
en: 1
del: 1
hace: 1
al: 1
es: 1
sea: 1
doy: 1
este: 1
se: 1
nuevo: 3
cada: 2
menos: 1
final: 1
paso: 6
un: 2
y: 2
largo: 2
empieza: 1
acorta: 1
```

Figura 3: Ejecución del analizador de palabras en consola.

```
Duplicadas sin ordenar:
que: 3
camino: 4
el: 2
nuevo: 3
cada: 2
paso: 6
un: 2
y: 2
largo: 2

Duplicadas en orden alfabético:
cada: 2
camino: 4
el: 2
largo: 2
nuevo: 3
paso: 6
que: 3
un: 2
y: 2
```

Figura 4: Ejecución del analizador de palabras en consola.

5. Conclusiones

La investigación sobre los destructores en Java permitió comprender que, a diferencia de otros lenguajes, la gestión de memoria no depende del programador, sino de un proceso automático como el Recolector de Basura. Esto evidencia la importancia de conocer mecanismos como `AutoCloseable` y `try-with-resources` para el manejo seguro de recursos externos, ya que la liberación de memoria y la liberación de recursos del sistema son procesos distintos que no deben confundirse.

Por otro lado, la implementación del analizador de palabras duplicadas demostró la utilidad práctica de las estructuras `HashMap` y `TreeSet` vistas en el Marco Teórico. El uso de `HashMap` permitió contar la frecuencia de cada palabra de forma eficiente mediante pares clave-valor, mientras que el `TreeSet` facilitó ordenar dichas palabras sin necesidad de implementar un algoritmo de ordenamiento manual. Las pruebas realizadas confirmaron que la aplicación normaliza correctamente el texto, ignorando mayúsculas y signos de puntuación, y que identifica de manera precisa tanto las palabras distintas como las duplicadas.

En conjunto, ambos ejercicios reforzaron la importancia de conocer las bibliotecas y utilerías del lenguaje Java, ya que permiten resolver problemas comunes de forma estructurada y eficiente, evitando implementar desde cero funcionalidades que ya están disponibles y probadas dentro del lenguaje. itemize

Referencias

- [1] L. Joyanes Aguilar y I. Zahonero Martínez, *Programación en Java 2: algoritmos, estructuras de datos y programación orientada a objetos*. Madrid: McGraw-Hill, 2002.
- [2] J. Dean y R. Dean, *Introducción a la programación con Java*, 1.a ed. México: McGraw-Hill, 2009.